P2P 네트워크 및 Mempool 아키텍처

목차

1. P2P 네트워크 개요
2. Peer Discovery
3. Reactor 패턴
4. Gossip 프로토콜
5. Mempool 아키텍처
6. Transaction Propagation
7. DoS 방어 메커니즘
8. State Sync
9. 성능 최적화
10. 모니터링 및 디버깅

---

1. P2P 네트워크 개요

1.1 CometBFT P2P 아키텍처

CometBFT는 **전이중 (full-duplex)**, **gossip 기반** P2P 네트워크를 사용합니다.

```
┌──────────────────────────────────────────────────┐
│                    Node A                        │
│                                                  │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐       │
│  │Consensus │  │ Mempool  │  │  Block   │       │
│  │ Reactor  │  │ Reactor  │  │ Reactor  │  ...  │
│  └──────────┘  └──────────┘  └──────────┘       │
│       │            │             │               │
│       └────────────┴─────────────┘               │
│                    │                             │
│              ┌─────────────┐                     │
│              │  P2P Switch │                     │
│              └─────────────┘                     │
└──────────────────────────────────────────────────┘
                    │
        ┌───────────┴───────────┐
        │           │           │
        ▼           ▼           ▼
    ┌───────┐   ┌───────┐   ┌───────┐
    │Node B │   │Node C │   │Node D │
    └───────┘   └───────┘   └───────┘
```

1.2 주요 컴포넌트

P2P Switch

- **역할**: 모든 P2P 통신의 중앙 허브
- **책임**: Peer 연결 관리, 메시지 라우팅

Reactor

- **역할**: 특정 프로토콜 구현 (Consensus, Mempool, Block Sync 등)
- **책임**: 메시지 처리, 데이터 동기화

Peer

- **역할**: 원격 노드와의 연결
- **책임**: 메시지 송수신, 연결 상태 관리

---

2. Peer Discovery

2.1 Bootstrap 노드 연결

config.toml 설정

```toml
# config.toml
[p2p]
# Seed nodes (초기 연결)
seeds = "node-id@ip:port,node-id@ip:port"

# Persistent peers (항상 유지)
persistent_peers = "node-id@ip:port"

# 최대 peer 수
max_num_inbound_peers = 40
max_num_outbound_peers = 10

# PEX (Peer Exchange) 활성화
pex = true

# Address book
addr_book_file = "addrbook.json"
addr_book_strict = true
```

2.2 PEX (Peer Exchange) 프로토콜

PEX: Peer들이 서로 알고 있는 Peer 목록을 교환하는 프로토콜

```go
// p2p/pex/pex_reactor.go
type Reactor struct {
    book              AddrBook          // Address book
    config            *PEXReactorConfig
    ensurePeersPeriod time.Duration     // 주기적으로 peer 확보
}

// Address book에서 Peer 요청
func (r *Reactor) RequestAddrs(peer Peer) {
    peer.Send(PexChannel, cdc.MustMarshalBinaryBare(&pexRequestMessage{}))
}

// Peer 목록 전송
func (r *Reactor) SendAddrs(peer Peer, addrs []*p2p.NetAddress) {
    msg := &pexAddrsMessage{Addrs: addrs}
    peer.Send(PexChannel, cdc.MustMarshalBinaryBare(msg))
}
```

PEX 프로토콜 플로우

```
Node A (새로 시작)
  ↓
1. config.toml의 seeds에 연결
  ↓
2. Seed에게 PexRequest 전송
  ↓
3. Seed가 PexAddrs 응답 (50개 peer 주소)
  ↓
4. Address Book에 저장
  ↓
5. 일부 주소로 연결 시도
  ↓
6. 연결된 Peer들에게도 주기적으로 PexRequest
  ↓
7. Address Book 지속적으로 업데이트
```

2.3 Address Book

```go
// p2p/pex/addrbook.go
type AddrBook struct {
    mtx               sync.Mutex
    rand              *rand.Rand
    ourAddrs          map[string]struct{}      // 자신의 주소
    privateIDs        map[p2p.ID]struct{}      // Private peer IDs
    addrLookup        map[p2p.ID]*knownAddress // ID → Address
    bucketsOld        []map[string]*knownAddress // Old addresses
    bucketsNew        []map[string]*knownAddress // New addresses
    nOld              int
    nNew              int
}

// Address 추가
func (a *AddrBook) AddAddress(addr *p2p.NetAddress, src *p2p.NetAddress) error {
    a.mtx.Lock()
    defer a.mtx.Unlock()

    // 1. 유효성 검사
    if !addr.Routable() {
        return ErrAddrBookNonRoutable{addr}
    }

    // 2. 이미 알고 있는지 확인
    ka := a.addrLookup[addr.ID]
    if ka != nil {
        // 이미 알고 있음 - 업데이트만
        ka.LastAttempt = time.Now()
        ka.Attempts++
        return nil
    }

    // 3. New bucket에 추가
    ka = &knownAddress{
        Addr:        addr,
        Src:         src,
        Attempts:    0,
        LastAttempt: time.Now(),
    }

    bucket := a.calcNewBucket(addr, src)
    a.bucketsNew[bucket][addr.String()] = ka
    a.addrLookup[addr.ID] = ka
    a.nNew++

    return nil
}

// 연결할 주소 선택
func (a *AddrBook) PickAddress(biasTowardsNewAddrs int) *p2p.NetAddress {
    a.mtx.Lock()
    defer a.mtx.Unlock()

    // New bucket vs Old bucket 선택 (확률적)
    if a.nNew > 0 && (a.nOld == 0 || a.rand.Intn(100) < biasTowardsNewAddrs) {
        // New bucket에서 선택
        return a.pickFromBucket(a.bucketsNew)
    }

    // Old bucket에서 선택
    return a.pickFromBucket(a.bucketsOld)
}
```

3. Reactor 패턴

3.1 Reactor 인터페이스

```go
// p2p/base_reactor.go
type Reactor interface {
    // GetChannels는 이 Reactor가 사용하는 채널 목록 반환
    GetChannels() []*ChannelDescriptor

    // SetSwitch는 Switch 참조 설정
    SetSwitch(*Switch)

    // OnStart는 Reactor 시작 시 호출
    OnStart() error

    // OnStop은 Reactor 종료 시 호출
    OnStop()

    // AddPeer는 새 peer 연결 시 호출
    AddPeer(peer Peer)

    // RemovePeer는 peer 연결 종료 시 호출
    RemovePeer(peer Peer, reason interface{})

    // Receive는 메시지 수신 시 호출
    Receive(chID byte, peer Peer, msgBytes []byte)
}
```

3.2 주요 Reactor들

Consensus Reactor

```go
// consensus/reactor.go
type Reactor struct {
    conS *State // Consensus state

    mtx      sync.RWMutex
    waitSync bool
    eventBus *types.EventBus

    // Metrics
    metrics *Metrics
}

// Receive 구현
func (conR *Reactor) Receive(chID byte, src Peer, msgBytes []byte) {
    msg, err := decodeMsg(msgBytes)
    if err != nil {
        conR.Logger.Error("Error decoding message", "src", src, "chId", chID, "err", err)
        return
    }

    switch chID {
    case StateChannel:
        switch msg := msg.(type) {
        case *NewRoundStepMessage:
            conR.conS.mtx.Lock()
            initialHeight := conR.conS.state.InitialHeight
            conR.conS.mtx.Unlock()
            if err = msg.ValidateHeight(initialHeight); err != nil {
                return
            }
            ps := conR.getPeerState(src.ID())
            ps.ApplyNewRoundStepMessage(msg)

        case *HasVoteMessage:
            ps := conR.getPeerState(src.ID())
            ps.ApplyHasVoteMessage(msg)

        // ... 기타 메시지 처리
        }

    case DataChannel:
        // Block parts, Proposal, ProposalPOL 등
    }
}

// Gossip vote
func (conR *Reactor) gossipVotesRoutine(peer Peer, ps *PeerState) {
    for {
        select {
        case <-time.After(peerGossipSleepDuration):
            // 주기적으로 Vote gossip
            conR.gossipVotesForHeight(ps)
        case <-peer.Quit():
            return
        }
    }
}
```

Mempool Reactor

```go
// mempool/reactor.go
type Reactor struct {
    config  *cfg.MempoolConfig
    mempool *CListMempool
    ids     *mempoolIDs // Peer별 전송한 tx 추적
}

// Receive 구현
func (memR *Reactor) Receive(chID byte, src Peer, msgBytes []byte) {
    msg, err := decodeMsg(msgBytes)
    if err != nil {
        return
    }

    switch msg := msg.(type) {
    case *TxMessage:
        // 트랜잭션 수신
        peerID := memR.ids.GetForPeer(src)
        if !peerID.Has(msg.Tx.Key()) {
            // 이 peer로부터 처음 받는 tx
            err := memR.mempool.CheckTx(msg.Tx, nil, TxInfo{SenderID: memR.ids.GetForPeer(src)})
            if err != nil {
                return
            }
        }
    }
}

// Broadcast tx
func (memR *Reactor) broadcastTxRoutine(peer Peer) {
    peerID := memR.ids.GetForPeer(peer)

    for {
        select {
        case <-time.After(peerCatchupSleepIntervalMS * time.Millisecond):
            // Peer가 아직 받지 못한 tx 전송
            memTx := memR.mempool.NextTxToSend(peerID)
            if memTx == nil {
                continue
            }

            msg := &TxMessage{Tx: memTx.tx}
            success := peer.Send(MempoolChannel, cdc.MustMarshalBinaryBare(msg))
            if success {
                peerID.MarkSent(memTx.tx.Key())
            }

        case <-peer.Quit():
            return
        }
    }
}
```

Block Sync Reactor (BlockPool)

```go
// blocksync/reactor.go
type Reactor struct {
    initialState sm.State
    blockExec    *sm.BlockExecutor
    store        sm.BlockStore
    pool         *BlockPool       // 블록 요청/수신 관리
    fastSync     bool
    requestsCh   chan<- BlockRequest
    errorsCh     <-chan peerError
}

// Block 요청
func (bcR *Reactor) requestRoutine() {
    for {
        request := bcR.pool.NextRequest()
        if request == nil {
            time.Sleep(requestIntervalMS * time.Millisecond)
            continue
        }

        peer := bcR.pool.PickPeer()
        if peer == nil {
            time.Sleep(requestIntervalMS * time.Millisecond)
            continue
        }

        // Block 요청 전송
        msg := &bcBlockRequestMessage{Height: request.Height}
        peer.Send(BlocksyncChannel, cdc.MustMarshalBinaryBare(msg))
    }
}
```

---

4. Gossip 프로토콜

4.1 Gossip의 필요성

문제: 모든 노드에 메시지를 전달하려면?

- ❌ **Broadcasting**: 모든 peer에게 직접 전송 → $O(N^2)$ 메시지
- ✅ **Gossiping**: 일부 peer에게만 전송, 재귀적으로 전파 → $O(N \log N)$ 메시지

4.2 Epidemic Gossip 알고리즘

```
Node A가 새 트랜잭션 수신:

Round 1:
  A → [B, C] (무작위 2개 peer 선택)

Round 2:
  B → [D, E]
  C → [F, G]

Round 3:
  D → [H, I]
  E → [J, K]
  F → [L, M]
  G → [N, O]

... 계속 전파

결과: log(N) 라운드에 모든 노드 도달
```

4.3 중복 방지 메커니즘

```go
// mempool/mempool.go
type mempoolIDs struct {
    mtx       sync.RWMutex
    peerMap   map[p2p.ID]uint16          // Peer ID → internal ID
    nextID    uint16
    activeIDs map[uint16]struct{}        // Active internal IDs
}

// Peer별로 어떤 tx를 전송했는지 추적
type PeerState struct {
    sent     map[types.TxKey]struct{}   // 이 peer에게 보낸 tx
    received map[types.TxKey]struct{}   // 이 peer로부터 받은 tx
}

func (ps *PeerState) MarkSent(txKey types.TxKey) {
    ps.mtx.Lock()
    defer ps.mtx.Unlock()
    ps.sent[txKey] = struct{}{}
}

func (ps *PeerState) HasSent(txKey types.TxKey) bool {
    ps.mtx.RLock()
    defer ps.mtx.RUnlock()
    _, ok := ps.sent[txKey]
    return ok
}
```

4.4 실전 Gossip 최적화

```go
// consensus/reactor.go
func (conR *Reactor) gossipVotesForHeight(rs *cstypes.RoundState) {
    // Square-root Gossip: O(√N) peer에게 전송
    peers := conR.Switch.Peers().List()
    numPeers := len(peers)
    numToSend := int(math.Sqrt(float64(numPeers))) + 1

    // 무작위로 peer 선택
    rand.Shuffle(len(peers), func(i, j int) {
        peers[i], peers[j] = peers[j], peers[i]
    })

    for i := 0; i < numToSend && i < numPeers; i++ {
        peer := peers[i]
        ps := peer.Get(types.PeerStateKey).(*PeerState)

        // 이 peer가 아직 받지 못한 vote 전송
        for _, vote := range rs.Votes.Prevotes(rs.Round).Votes {
            if vote != nil && !ps.HasVote(vote.Height, vote.Round, vote.Type, vote.ValidatorIndex) {
                peer.Send(VoteChannel, MustEncode(&VoteMessage{Vote: vote}))
            }
        }
    }
}
```

5. Mempool 아키텍처

5.1 Mempool 구조

```go
// mempool/clist_mempool.go
type CListMempool struct {
    height        int64              // 현재 블록 높이
    txsBytes      int64              // 총 메모리 사용량
    txs           *clist.CList       // 트랜잭션 연결 리스트
    txsMap        sync.Map           // TxKey → *clistElement
    cache         TxCache            // 중복 방지 캐시
    config        *cfg.MempoolConfig
    proxyAppConn  proxy.AppConnMempool
    metrics       *Metrics

    // 동시성 제어
    updateMtx     sync.RWMutex
    preCheck      PreCheckFunc       // CheckTx 전 검사
    postCheck     PostCheckFunc      // CheckTx 후 검사
}

// Mempool에 저장되는 트랜잭션
type mempoolTx struct {
    height    int64      // 추가된 블록 높이
    gasWanted int64      // 필요한 gas
    tx        types.Tx   // 실제 트랜잭션
    sender    string     // 발신자
}
```

5.2 CheckTx 플로우

```go
// mempool/clist_mempool.go
func (mem *CListMempool) CheckTx(
    tx types.Tx,
    callback func(*abci.Response),
    txInfo TxInfo,
) error {
    // 1. 크기 검사
    if mem.config.MaxTxBytes > 0 && len(tx) > mem.config.MaxTxBytes {
        return ErrTxTooLarge{
            Max:    mem.config.MaxTxBytes,
            Actual: len(tx),
        }
    }

    // 2. Mempool 크기 검사
    if mem.config.Size > 0 && mem.txs.Len() >= mem.config.Size {
        return ErrMempoolIsFull{
            NumTxs:  mem.txs.Len(),
            MaxTxs:  mem.config.Size,
        }
    }

    // 3. 메모리 사용량 검사
    if mem.config.MaxTxsBytes > 0 && mem.txsBytes+int64(len(tx)) > mem.config.MaxTxsBytes {
        return ErrMempoolIsFull{
            NumTxs:  mem.txs.Len(),
            MaxTxs:  mem.config.Size,
        }
    }

    // 4. 중복 검사 (Cache)
    if !mem.cache.Push(tx) {
        return ErrTxInCache
    }

    // 5. PreCheck (설정된 경우)
    if mem.preCheck != nil {
        if err := mem.preCheck(tx); err != nil {
            return err
        }
    }

    // 6. ABCI CheckTx 호출
    reqRes := mem.proxyAppConn.CheckTxAsync(abci.RequestCheckTx{Tx: tx})
    reqRes.SetCallback(func(res *abci.Response) {
        mem.reqResCb(tx, txInfo.SenderID, txInfo.SenderP2PID, res, callback)
    })

    return nil
}

// CheckTx 응답 처리
func (mem *CListMempool) reqResCb(
    tx types.Tx,
    peerID uint16,
    peerP2PID p2p.ID,
    res *abci.Response,
    cb func(*abci.Response),
) {
    checkTxRes := res.GetCheckTx()

    if checkTxRes.Code == abci.CodeTypeOK {
        // 성공 - Mempool에 추가
        memTx := &mempoolTx{
            height:    mem.height,
            gasWanted: checkTxRes.GasWanted,
            tx:        tx,
        }

        mem.updateMtx.Lock()
        e := mem.txs.PushBack(memTx)
        mem.txsMap.Store(tx.Key(), e)
        mem.txsBytes += int64(len(tx))
        mem.updateMtx.Unlock()

        mem.metrics.Size.Set(float64(mem.txs.Len()))
```

```
        mem.metrics.TxSizeBytes.Observe(float64(len(tx)))
    } else {
        // 실패 - Cache에서 제거
        mem.cache.Remove(tx)
    }

    if cb != nil {
        cb(res)
    }
}
```

5.3 Mempool Recheck

블록이 커밋되면 mempool의 트랜잭션들을 재검증합니다:

```go
// mempool/clist_mempool.go
func (mem *CListMempool) Update(
    height int64,
    txs types.Txs,
    deliverTxResponses []*abci.ExecTxResult,
    preCheck PreCheckFunc,
    postCheck PostCheckFunc,
) error {
    mem.updateMtx.Lock()
    defer mem.updateMtx.Unlock()

    // 1. 블록에 포함된 tx 제거
    for _, tx := range txs {
        if e, ok := mem.txsMap.Load(tx.Key()); ok {
            mem.removeTx(tx, e.(*clist.CElement), false)
        }
    }

    // 2. Height 업데이트
    mem.height = height

    // 3. PreCheck / PostCheck 업데이트
    mem.preCheck = preCheck
    mem.postCheck = postCheck

    // 4. Recheck (선택적)
    if mem.config.Recheck {
        mem.recheckTxs()
    }

    // 5. ABCI Mempool 통지
    if mem.Size() > 0 {
        mem.notifyTxsAvailable()
    }

    return nil
}

// 모든 트랜잭션 재검증
func (mem *CListMempool) recheckTxs() {
    if mem.Size() == 0 {
        return
    }

    for e := mem.txs.Front(); e != nil; e = e.Next() {
        memTx := e.Value.(*mempoolTx)

        // ABCI CheckTx(Recheck) 호출
        reqRes := mem.proxyAppConn.CheckTxAsync(abci.RequestCheckTx{
            Tx:   memTx.tx,
            Type: abci.CheckTxType_Recheck,
        })

        reqRes.SetCallback(func(res *abci.Response) {
            checkTxRes := res.GetCheckTx()
            if checkTxRes.Code != abci.CodeTypeOK {
                // 더 이상 유효하지 않음 - 제거
                mem.removeTx(memTx.tx, e, false)
            }
        })
    }

    mem.proxyAppConn.FlushAsync()
}
```

6. Transaction Propagation

6.1 트랜잭션 전파 플로우

```
1. Client → Node A
   ↓
2. Node A: CheckTx (ABCI)
   ↓
3. CheckTx 성공 → Mempool 추가
   ↓
4. Mempool Reactor: Broadcast
   ↓
5. Node B, C, D: 트랜잭션 수신
   ↓
6. 각 노드: CheckTx → Mempool 추가
   ↓
7. 재귀적으로 전파
   ↓
8. 전체 네트워크에 전파 완료 (수초 내)
```

6.2 Priority Mempool (향후 기능)

```go
// Priority에 따른 정렬
type PriorityMempool struct {
    txs *priorityQueue // Min-heap by priority
}

type priorityQueue struct {
    items []*mempoolTx
}

func (pq *priorityQueue) Less(i, j int) bool {
    // Priority 높은 것이 먼저 (Max-heap처럼 동작)
    return pq.items[i].priority > pq.items[j].priority
}

// Reap: Priority 순으로 반환
func (mem *PriorityMempool) ReapMaxBytesMaxGas(maxBytes, maxGas int64) types.Txs {
    heap.Init(&mem.txs)

    var (
        totalBytes int64
        totalGas   int64
        txs        []types.Tx
    )

    for mem.txs.Len() > 0 {
        memTx := heap.Pop(&mem.txs).(*mempoolTx)

        if totalGas+memTx.gasWanted > maxGas {
            continue
        }
        if totalBytes+int64(len(memTx.tx)) > maxBytes {
            break
        }

        totalGas += memTx.gasWanted
        totalBytes += int64(len(memTx.tx))
        txs = append(txs, memTx.tx)
    }

    return txs
}
```

7. DoS 방어 메커니즘

7.1 Rate Limiting

```go
// p2p/conn/connection.go
type MConnection struct {
    conn        net.Conn
    channels    []*Channel
    sendMonitor *flow.Monitor  // 송신 모니터링
    recvMonitor *flow.Monitor  // 수신 모니터링
    sendRate    int64          // Bytes per second
    recvRate    int64
}

// Channel별 Rate Limiting
type Channel struct {
    id               byte
    sendQueue        chan []byte
    sendQueueSize    int
    recvQueue        chan []byte
    recvQueueSize    int
    maxPacketMsgSize int
    priority         int
}

// 메시지 전송 (Rate limit 적용)
func (c *MConnection) Send(chID byte, msgBytes []byte) bool {
    channel, ok := c.channelsIdx[chID]
    if !ok {
        return false
    }

    // Rate limit 체크
    if !c.sendMonitor.Update(len(msgBytes)) {
        c.Logger.Debug("Send rate exceeded", "chID", chID)
        return false
    }

    select {
    case channel.sendQueue <- msgBytes:
        return true
    default:
        // Queue full
        return false
    }
}
```

7.2 Peer Scoring

```go
// p2p/peer.go
type Peer struct {
    nodeInfo         NodeInfo
    mconn            *MConnection
    persistent       bool

    // Peer quality metrics
    lastReceivedTime time.Time
    lastSendTime     time.Time
    sendQueue        chan Envelope
    score            int // 점수 (낮을수록 나쁨)
}

// 나쁜 peer 판단
func (p *Peer) IsBad() bool {
    // 1. 오랫동안 메시지 없음
    if time.Since(p.lastReceivedTime) > peerTimeout {
        return true
    }

    // 2. Score가 너무 낮음
    if p.score < minPeerScore {
        return true
    }

    // 3. Send queue 계속 full
    if len(p.sendQueue) == cap(p.sendQueue) {
        return true
    }

    return false
}

// Peer 제거
func (sw *Switch) StopPeerForError(peer Peer, reason interface{}) {
    sw.Logger.Error("Stopping peer for error", "peer", peer, "reason", reason)
    sw.StopPeerGracefully(peer)

    // Address book에서 제거 또는 penalty
    if sw.addrBook != nil {
        sw.addrBook.MarkBad(peer.NodeInfo().NetAddress(), defaultBanTime)
    }
}
```

7.3 Mempool DoS 방어

```go
// mempool/clist_mempool.go
type CListMempool struct {
    config *cfg.MempoolConfig
}

type MempoolConfig struct {
    // 크기 제한
    Size         int   // 최대 트랜잭션 개수
    MaxTxsBytes  int64 // 최대 메모리 사용량
    MaxTxBytes   int   // 단일 트랜잭션 최대 크기

    // Cache
    CacheSize int // 중복 방지 캐시 크기

    // Broadcasting
    Broadcast bool // 다른 peer에게 broadcast 여부
}

// 공격 방어
func (mem *CListMempool) CheckTx(tx types.Tx, callback func(*abci.Response), txInfo TxInfo) error {
    // 1. 너무 큰 트랜잭션 거부
    if len(tx) > mem.config.MaxTxBytes {
        return ErrTxTooLarge
    }

    // 2. Mempool Full → 거부
    if mem.txs.Len() >= mem.config.Size {
        return ErrMempoolIsFull
    }

    // 3. 중복 트랜잭션 거부 (Cache)
    if !mem.cache.Push(tx) {
        return ErrTxInCache
    }

    // 4. PreCheck: Gas price 등 사전 검증
    if mem.preCheck != nil {
        if err := mem.preCheck(tx); err != nil {
            mem.cache.Remove(tx)
            return err
        }
    }

    // ... CheckTx 계속
}
```

---

8. State Sync

8.1 State Sync 개요

문제: 새 노드가 Genesis부터 모든 블록을 실행하면 너무 오래 걸림

해결: State Sync - 최근 상태 스냅샷을 다운로드

```
Traditional Sync (Block Sync):
  Genesis → Block 1 → Block 2 → ... → Block 1,000,000
  시간: 수일 ~ 수주

State Sync:
  Snapshot at Block 999,000 → Block 999,001 → ... → Block 1,000,000
  시간: 수분 ~ 수시간
```

8.2 State Sync 프로토콜

```go
// statesync/reactor.go
type Reactor struct {
    stateProvider StateProvider    // 스냅샷 제공자
    conn          proxy.AppConnSnapshot
    tempDir       string
    snapshotCh    *p2p.Channel
    chunkCh       *p2p.Channel
    peerUpdates   *p2p.PeerUpdatesCh
}

// Snapshot discovery
func (r *Reactor) discoverSnapshots() ([]*snapshot, error) {
    snapshots := make(map[p2p.ID]*snapshot)

    // 모든 peer에게 SnapshotsRequest 전송
    for _, peer := range r.Switch.Peers().List() {
        peer.Send(SnapshotChannel, cdc.MustMarshalBinaryBare(&SnapshotsRequest{}))
    }

    // 응답 수집
    timeout := time.After(snapshotDiscoveryTimeout)
    for {
        select {
        case <-timeout:
            goto DONE
        case msg := <-r.snapshotCh:
            switch msg := msg.(type) {
            case *SnapshotsResponse:
                for _, snapshot := range msg.Snapshots {
                    snapshots[msg.PeerID] = snapshot
                }
            }
        }
    }

DONE:
    return snapshots, nil
}

// Snapshot chunks 다운로드
func (r *Reactor) fetchChunks(snapshot *snapshot) error {
    numChunks := snapshot.Chunks

    for i := uint32(0); i < numChunks; i++ {
        peer := r.pickPeer(snapshot)
        if peer == nil {
            return errors.New("no peers available")
        }

        // Chunk 요청
        peer.Send(ChunkChannel, cdc.MustMarshalBinaryBare(&ChunkRequest{
            Height: snapshot.Height,
            Format: snapshot.Format,
            Index:  i,
        }))

        // Chunk 수신
        chunk := <-r.chunkCh
        if err := r.conn.ApplySnapshotChunk(chunk); err != nil {
            return err
        }
    }

    return nil
}
```

8.3 Application State Sync 구현

```go
// baseapp/abci.go
func (app *BaseApp) OfferSnapshot(req *abci.RequestOfferSnapshot) (*abci.ResponseOfferSnapshot, error) {
    // Snapshot 수락 여부 결정
    if req.Snapshot.Height < app.initialHeight {
        return &abci.ResponseOfferSnapshot{Result: abci.ResponseOfferSnapshot_REJECT}, nil
    }

    return &abci.ResponseOfferSnapshot{Result: abci.ResponseOfferSnapshot_ACCEPT}, nil
}

func (app *BaseApp) ApplySnapshotChunk(req *abci.RequestApplySnapshotChunk) (*abci.ResponseApplySnapshotChunk, error) {
    // Chunk 적용
    err := app.cms.ApplySnapshotChunk(req.Index, req.Chunk)
    if err != nil {
        return &abci.ResponseApplySnapshotChunk{
            Result: abci.ResponseApplySnapshotChunk_RETRY,
        }, nil
    }

    return &abci.ResponseApplySnapshotChunk{
        Result: abci.ResponseApplySnapshotChunk_ACCEPT,
    }, nil
}
```

---

9. 성능 최적화

9.1 Connection Pooling

```go
// p2p/conn_set.go
type ConnSet struct {
    conns map[string]net.Conn
    mtx   sync.Mutex
}

func (cs *ConnSet) Get(addr string) net.Conn {
    cs.mtx.Lock()
    defer cs.mtx.Unlock()
    return cs.conns[addr]
}

func (cs *ConnSet) Set(addr string, conn net.Conn) {
    cs.mtx.Lock()
    defer cs.mtx.Unlock()
    cs.conns[addr] = conn
}
```

9.2 Message Batching

```go
// consensus/reactor.go
const (
    maxMsgSize   = 1048576 // 1 MB
    batchTimeout = 10 * time.Millisecond
)

type messageBatch struct {
    msgs []Message
    size int
}

func (conR *Reactor) batchSend(peer Peer, msgs []Message) {
    var batch messageBatch

    for _, msg := range msgs {
        msgSize := len(cdc.MustMarshalBinaryBare(msg))

        // Batch 크기 초과 시 전송
        if batch.size+msgSize > maxMsgSize {
            peer.Send(StateChannel, cdc.MustMarshalBinaryBare(&batch))
            batch = messageBatch{}
        }

        batch.msgs = append(batch.msgs, msg)
        batch.size += msgSize
    }

    // 남은 메시지 전송
    if len(batch.msgs) > 0 {
        peer.Send(StateChannel, cdc.MustMarshalBinaryBare(&batch))
    }
}
```

9.3 Parallel Processing

```go
// mempool/clist_mempool.go
func (mem *CListMempool) CheckTxParallel(txs types.Txs) []error {
    errors := make([]error, len(txs))
    var wg sync.WaitGroup

    // Worker pool
    numWorkers := runtime.NumCPU()
    txChan := make(chan struct {
        index int
        tx    types.Tx
    }, len(txs))

    // Workers
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for item := range txChan {
                errors[item.index] = mem.CheckTx(item.tx, nil, TxInfo{})
            }
        }()
    }

    // Submit tasks
    for i, tx := range txs {
        txChan <- struct {
            index int
            tx    types.Tx
        }{i, tx}
    }
    close(txChan)

    wg.Wait()
    return errors
}
```

10. 모니터링 및 디버깅

10.1 Prometheus Metrics

```go
// p2p/metrics.go
type Metrics struct {
    Peers                  metrics.Gauge   // 연결된 peer 수
    PeersConnecting        metrics.Gauge   // 연결 중인 peer 수
    PeerReceiveBytesTotal  metrics.Counter // 수신 바이트
    PeerSendBytesTotal     metrics.Counter // 송신 바이트
    MessageReceiveTotal    metrics.Counter // 수신 메시지 수
    MessageSendTotal       metrics.Counter // 송신 메시지 수
}

// 예시: Metrics 업데이트
func (sw *Switch) addPeer(peer Peer) {
    sw.peers.Add(peer)
    sw.metrics.Peers.Set(float64(sw.peers.Size()))
}
```

10.2 RPC 모니터링

```
# Peer 목록
$ curl http://localhost:26657/net_info

{
  "n_peers": "10",
  "peers": [
    {
      "node_info": {
        "id": "abc123...",
        "listen_addr": "tcp://0.0.0.0:26656",
        "network": "cosmoshub-4",
        "version": "0.34.24",
        "channels": "4020212223303800",
        "moniker": "my-node"
      },
      "is_outbound": true,
      "connection_status": {
        "Duration": "12345678",
        "SendMonitor": {
          "Active": true,
          "Start": "2023-01-01T00:00:00Z",
          "Bytes": "1234567",
          "Samples": "1000",
          "InstRate": "12345",
          "CurRate": "12000",
          "AvgRate": "10000",
          "PeakRate": "15000"
        },
        "RecvMonitor": { ... }
      }
    },
    ...
  ]
}

# Mempool 상태
$ curl http://localhost:26657/num_unconfirmed_txs

{
  "n_txs": "42",
  "total": "42",
  "total_bytes": "123456"
}

# Consensus 상태
$ curl http://localhost:26657/consensus_state

{
  "height": "12345",
  "round": "0",
  "step": "RoundStepPrevote",
  "start_time": "2023-01-01T00:00:00Z",
  "commit_time": "2023-01-01T00:00:06Z",
  "validators": [ ... ],
  "proposal": { ... },
  "locked_block": null,
  "valid_block": null
}
```

참고 자료

공식 문서

- CometBFT P2P Spec
- Mempool Spec
- State Sync

코드

- p2p/
- mempool/

다음 단계

다음 문서에서는 **성능 분석 및 벤치마킹**을 다룹니다:

- 프로파일링 도구
- 병목 지점 분석
- 최적화 기법
- 벤치마크 결과

→ 11-performance-analysis.md